OBJECT ORIENTED PROGRAMMING THROUGH C++

UNIT 5

1 Explain the principles of exception handling in C++ with a simple flow example.

2 What are multiple catch statements? Explain with an example

3 Describe the role of try, throw, and catch in exception handling with syntax.

4 Explain the steps to open, write, read, and close a file in C++ with sample code.

5 What is STL? Explain its components and programming model.

6 What are iterators in STL? Explain their types and usage.

7 Explain vectors, Lists, and maps in STL with syntax and suitable example.

1.Explain the principles of exception handling in C++ with a simple flow example.

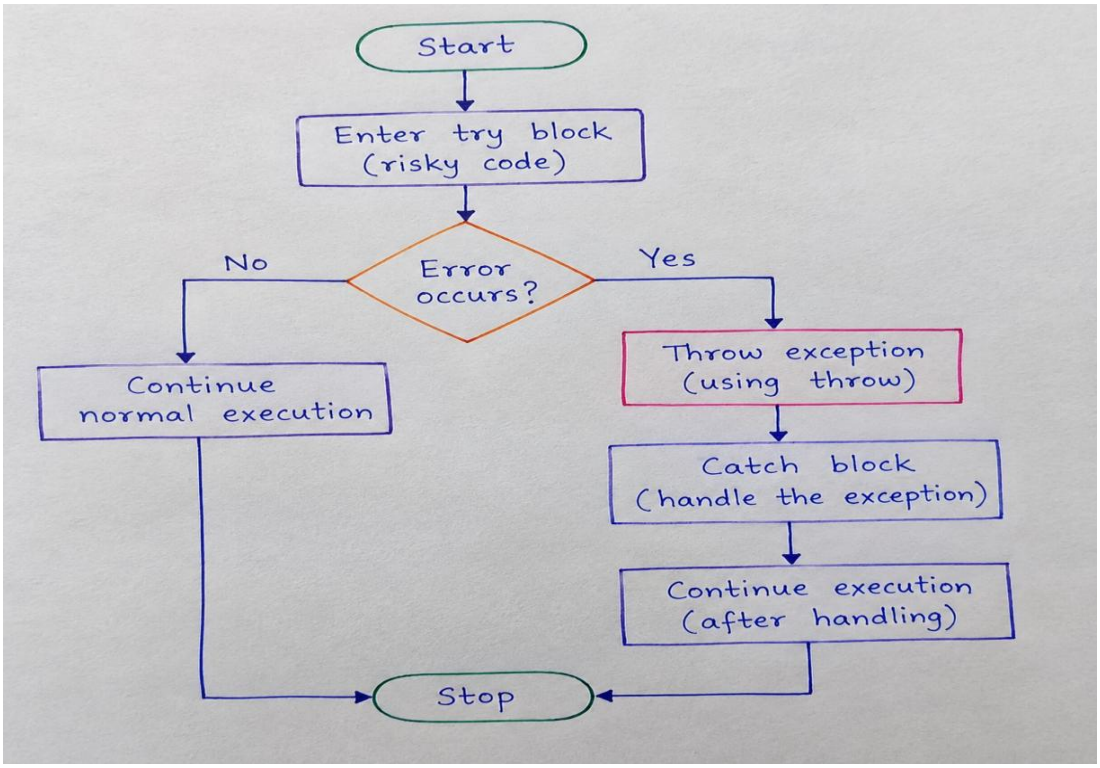
Definition

Exception handling is a mechanism in C++ used to handle runtime errors without abruptly terminating the program.

Principles of Exception Handling

1. Try – The code that may cause an error is placed inside a try block.
2. Throw – When an error occurs, an exception is generated using throw.
3. Catch – The catch block receives and handles the exception.
4. Normal execution – After handling the exception, the program can continue normally.

Simple Flow



Example

```
#include <iostream>
using namespace std;
int main() {
   try {
      int a = 10, b = 0;
      if (b == 0)
         throw "Division by zero";
      cout << a / b;
   }
   catch (const char* msg) {
      cout << "Exception: " << msg;
   }
   return 0;
}
```

Conclusion:

Thus, exception handling makes programs safer, reliable, and easier to debug.

---

2. What are multiple catch statements? Explain with an example.

Definition

Multiple catch statements are used when a program can generate different types of exceptions. Each exception type can have a separate catch block.

Syntax

```
try {
   // statements
}
catch (int e) {
   // handles integer exception
}
catch (double e) {
   // handles double exception
}
catch (const char* e) {
   // handles string exception
}
```

Example

```
#include <iostream>
using namespace std;
int main() {
   try {
      int choice = 1;

      if (choice == 1)
         throw 10;
      else
         throw 5.5;
   }
```

```
    catch (int e) {
        cout << "Integer exception: " << e;
    }
    catch (double e) {
        cout << "Double exception: " << e;
    }
    return 0;
}
```

Output

```
Integer exception: 10
```

Important Points

- A single try block can have multiple catch blocks.
- Each catch handles a different type of exception.
- Only the matching catch block is executed.
- Multiple catches make exception handling more flexible.

---

3. Describe the role of try, throw, and catch in exception handling with syntax.

C++ exception handling mainly uses three keywords: try, throw, and catch.

1. try

The try block contains code that may produce an exception.

Syntax:

```
try {
    // risky code
}
```

2. throw

The throw statement is used to generate an exception.

Syntax:

```
throw value;
```

3. catch

The catch block receives and handles the exception thrown by the throw statement.

Syntax:

```
catch(type variable) {
  // handling code
}
```

Example

```
#include <iostream>
using namespace std;
int main() {
  try {
    int age = 15;
    if (age < 18)
      throw age;
  }
  catch (int x) {
    cout << "Not eligible";
  }
  return 0;
}
```

Flow

try → throw → catch

Conclusion

try identifies risky code, throw generates an exception, and catch handles the exception.

---

4. Explain the steps to open, write, read, and close a file in C++ with sample code.

Definition

File handling in C++ is used to store and retrieve data from files permanently.

C++ provides:

- ofstream → writing to a file
- ifstream → reading from a file
- fstream → both reading and writing

Steps

1. Include <fstream>
2. Open the file
3. Write or read data
4. Close the file

Example

```
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
```

```
    // Open file for writing
    ofstream fout("data.txt");
    fout << "Hello C++";
    fout.close();
    // Open file for reading
    ifstream fin("data.txt");
    string text;
    getline(fin, text);
    cout << text;
    fin.close();
    return 0;
}
```

Explanation

Operation	Class/Function
Open for writing	ofstream
Write	<<
Open for reading	ifstream
Read	getline() / >>
Close	close()

Conclusion

File handling allows data to be stored permanently and accessed whenever required.

---

5. What is STL? Explain its components and programming model.

Definition

STL (Standard Template Library) is a collection of predefined generic classes and functions in C++ used for common data structures and algorithms.

Main Components of STL

1. Containers – Store and organize data.
    - vector
    - list
    - map
    - set
2. Algorithms – Perform operations on data.
    - sort()
    - find()
    - reverse()
3. Iterators – Used to access and move through container elements.
4. Function Objects – Objects that can be used like functions.

STL Programming Model:

Container

↓

Iterator

↓

Algorithm

↓

Data Processing

Example

```
#include <iostream>
#include <vector>
```

```
#include <algorithm>
using namespace std;
int main() {
   vector<int> v = {30, 10, 20};
   sort(v.begin(), v.end());
   for (int x : v)
      cout << x << " ";
   return 0;
}
```

Output

```
10 20 30
```

Conclusion

STL reduces programming effort by providing ready-made and reusable components.

---

6. What are iterators in STL? Explain their types and usage.

Definition

An iterator is an object used to point to and access elements of an STL container. It works similar to a pointer.

Basic Syntax

```
vector<int>::iterator it;
```

Common Types of Iterators

1. Input Iterator – Used to read elements.
2. Output Iterator – Used to write elements.
3. Forward Iterator – Moves only in the forward direction.
4. Bidirectional Iterator – Moves forward and backward.

5. Random Access Iterator – Can directly access any element.

Example

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
   vector<int> v = {10, 20, 30};
   vector<int>::iterator it;
   for (it = v.begin(); it != v.end(); it++) {
      cout << *it << " ";
   }
   return 0;
}
```

Output

```
10 20 30
```

Important Functions

- begin() → points to the first element
- end() → points just after the last element
- *it → accesses the current element
- ++it → moves iterator forward

Conclusion

Iterators provide a common way to access and process elements of STL containers.

7.Explain vectors, lists, and maps in STL with syntax and suitable example.

C++ STL provides different containers for storing data. Vector, List, and Map are commonly used containers.

1. Vector

A vector is a dynamic array that stores elements in continuous memory.

Syntax

```
vector<int> v;
```

Example

```
vector<int> v = {10, 20, 30};

v.push_back(40);

for (int x : v)

   cout << x << " ";
```

Output:

```
10 20 30 40
```

2. List

A list is a doubly linked list. It allows efficient insertion and deletion from the beginning and end.

Syntax

```
list<int> l;
```

Example

```
list<int> l = {10, 20, 30};

l.push_front(5);

l.push_back(40);

for (int x : l)

   cout << x << " ";
```

Output:

5 10 20 30 40

3. Map

A map stores data in key-value pairs. Each key is unique.

Syntax

```
map<int, string> m;
```

Example

```
map<int, string> m;
m[1] = "C++";
m[2] = "Python";
for (auto x : m)
    cout << x.first << " " << x.second << endl;
```

Output:

1 C++

2 Python

Difference

Container	Stores	Main Feature
Vector	Elements	Dynamic array
List	Elements	Easy insertion/deletion
Map	Key-value pairs	Unique keys

Conclusion

vector, list, and map are useful STL containers for storing and managing data in different ways.